

Agent Retrieval Design: When and Why Semantic Retrieval Is Needed

Author: Claus Albuquerque

Course: Agentic AI Program: Building Autonomous Systems for Real-World Applications

Date: August 8, 2026.

Is Retrieval Required?

Yes at recommendation rendering step. The core analytical reasoning does not need it.

The FinOps and SRE agents reason over structured, FOCUS-normalized data in PostgreSQL via typed tool calls — `query_cost_by_service(service_category='Compute', date_range='last_14d')` resolves to a SQL `WHERE` clause. The agent knows exactly what it needs and constructs precise queries. This is deterministic tool-based retrieval, not RAG.

Semantic retrieval becomes necessary when the agent must translate a cloud-agnostic conclusion into a **provider-specific, actionable recommendation**. The reasoning produces "this resource is overprovisioned; reducing capacity by 50% saves ~\$142/month." To be actionable for a GCP user, it must reference specific machine types (`n2-standard-4`), CLI commands (`gcloud compute instances set-machine-type`), and pricing details. This provider-specific knowledge is unstructured (documentation, pricing pages), broad (hundreds of SKUs), and context-dependent (relevant subset varies by resource type and region) — the agent cannot predict which document chunk it needs until analytical reasoning is complete.

Cost recommendations affect infrastructure, as such, they require **auditability** (traceable to a versioned document) and **determinism** (same query, same results). Web search provides neither. Instead, web scraping feeds the knowledge base update pipeline, while the agent retrieves from the curated local store during reasoning.

Semantic Retrieval Integration

The knowledge base contains **official GCP documentation**: machine type specifications, pricing models (on-demand, CUDs, sustained use discounts), `gcloud` CLI references, and operational constraints (live migration, maintenance windows). Documents are tagged by provider and resource type. As additional providers are added, each gets a separate partition in the same store.

The retrieval pipeline uses **pgvector** (PostgreSQL vector extension), keeping the knowledge base alongside operational data in the existing database. The retrieval tool `retrieve_provider_context(provider, resource_type, query, top_k=5)` operates in two stages:

1. **Metadata filter:** Narrows candidates by `provider='GCP' AND resource_type='compute/instance'` , eliminating cross-provider false matches and reducing the search space by ~80%.
2. **Vector similarity:** Cosine search within the filtered subset, returning the top 5 chunks with similarity scores.

Chunking strategy: Documents split at section headers (H2/H3), 512-token size constraint, 50-token overlap. Each chunk carries metadata: `provider` , `resource_type` , `category` , `last_verified` , and

`source_url` . Five retrieved chunks (~2,560 tokens) provide sufficient context without overwhelming the LLM's context window.

Example: How Retrieval Changes the Output

Without retrieval, the agent produces:

"Resource `analytics-worker-02` is overprovisioned. CPU: 12%, memory: 18% over 30 days. Reduce capacity by ~50%. Estimated savings: ~\$142/month."

Correct but not actionable — the engineer must research machine types independently.

With retrieval, the agent calls `retrieve_provider_context(provider='GCP', resource_type='compute/instance', query='n2 machine type 4 vCPUs 16GB')` and receives chunks containing the N2 specs table, the `set-machine-type` CLI reference, and sustained use discount details. It synthesizes:

*"VM `analytics-worker-02` has averaged 12% CPU and 18% memory over 30 days. **Downgrade from `n2-standard-8` (\$0.1942/hr) to `n2-standard-4` (\$0.0971/hr)**. Savings: \$142/month after sustained use discount. Note: instance must be stopped before resize. SRE confirms no availability group — brief downtime acceptable. [Approve] [Dismiss]"*

Retrieval transformed a generic conclusion into a specific, executable recommendation with exact SKU, pricing, operational constraints, and the CLI path.

Key Design Choices

1. **Official documentation only:** no community content, Stack Overflow, or third-party aggregators. Accuracy and auditability over breadth.
2. **Two-stage retrieval:** (metadata filter → vector search) prevents cross-provider contamination and improves precision.
3. **512-token chunks at section boundaries:** preserves semantic coherence (spec tables stay intact) while keeping chunks specific enough for meaningful similarity.
4. **Same embedding model for indexing and queries:** `nomic-embed-text-v1.5` consistency is critical for cosine similarity.
5. **Weekly automated update pipeline:** scrapes source docs, diffs against stored chunks, re-embeds changed sections, soft-deletes removed content (preserving audit trail).

Failure Mode: Stale Knowledge

The primary retrieval risk is **stale knowledge**, outdated chunks recommending deprecated SKUs or incorrect pricing. This is insidious because retrieval *appears* successful (high-similarity chunk returned) and the agent cannot distinguish stale from current content by text alone.

Mitigations:

- **last_verified filtering:** Chunks older than 30 days are excluded before the agent sees them.
- **Freshness scoring:** Chunks not re-verified in 14+ days receive a score penalty (similarity × `freshness_factor` decaying to 0.7), biasing toward recent content.
- **Post-execution feedback:** When executed recommendations produce costs diverging >10% from estimates, affected chunks are queued for re-verification — a self-healing loop.
- **Transparency:** For savings >\$1,000/month, the agent appends: "Verify current rates at [source_url] before approving."

Residual risk: price changes between weekly runs can produce incorrect estimates for up to 6 days. The design compensates with transparency rather than claiming real-time accuracy.